

Recovering Drift Fields from Random Walks

Framework Reference

The `dfi` package: data you can create, estimators you can plug in,
and the measurements that compare them

Version 0.1 • 6 600 lines • 28 tests passing

What this document is. A reference for the parts of the project that are already built and tested: how to *generate* flow fields, how to *simulate* walkers in them, what *data objects* come out, what *contract* an estimator must satisfy, and how errors are *measured* and *plotted*. The five estimators themselves are specified here but left unimplemented — they are yours to write. Sections marked **BUILT** work now; sections marked **YOURS** are stubs with full specifications.

Contents

1	The problem	2	4.3	Rung 1 — Binned Kramers–Moyal	10
2	The framework at a glance	2	4.4	Rung 2 — Kernel regression	11
2.1	Module map	3	4.5	Rung 3 — Basis projection (Stochastic Force Inference)	11
3	Data you can create	4	4.6	Rung 4 — Neural network regression . . .	11
3.1	The construction behind every random field	4	4.7	Rung 5 — Score-based, from snapshots . .	12
3.2	Field constructors	5	5	Measuring and comparing	13
3.3	Domains	6	5.1	Where you evaluate matters more than the metric	13
3.4	Trajectories: <code>simulate()</code>	6	5.2	What <code>drift_error</code> returns	13
3.5	What a Trajectories object holds	7	5.3	The error decomposition	13
3.6	Snapshots	8	5.4	Sweeps	13
3.7	Measurement noise	8	6	Figures and animations	15
3.8	A field’s own accessors	8	7	Design decisions, and the measurements be- hind them	17
4	Estimators	10	7.1	Validation already performed	17
4.1	The contract	10	8	Cheat sheet	19
4.2	The ladder	10			

1 The problem

We observe particles obeying the overdamped Langevin equation

$$dx = b(x) dt + \sqrt{2D} dW, \quad (1)$$

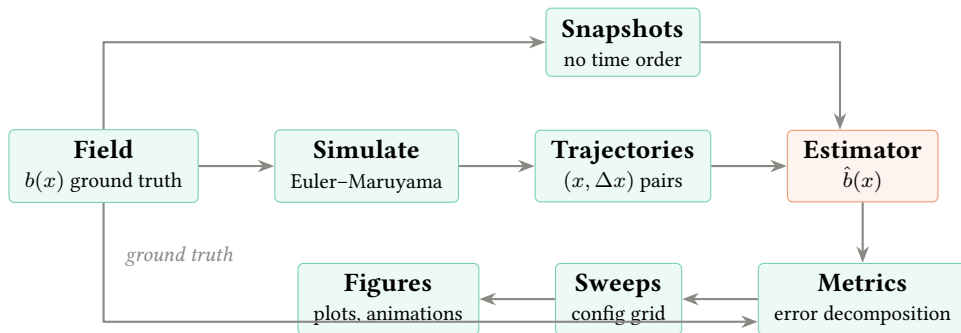
whose density evolves by the Fokker–Planck equation $\partial_t \rho = -\nabla \cdot (b\rho) + D\nabla^2 \rho$. The inverse problem is to recover the vector field $b(x)$ from observed motion.

There are two fundamentally different data regimes, and keeping them apart is the scientific spine of the project.

(a) Trajectories	(b) Snapshots
Positions <i>with</i> time ordering, sampled at spacing Δt . The Kramers–Moyal estimator is a local conditional average,	Unordered positions drawn from ρ_{ss} . The only recoverable object is the score,
$b(x) \approx \frac{1}{\Delta t} \mathbb{E}[\Delta x \mid x]$.	$\nabla \log \rho_{ss} = -\nabla U / D$.
Can recover <i>all</i> of b .	Can recover <i>only</i> the gradient part.

The identifiability gap. Split the drift into a curl-free and a divergence-free part, $b = -\nabla U + b_{\text{rot}}$. The rotational component carries a steady probability current but leaves ρ_{ss} *unchanged*. So a snapshot dataset is *literally identical in distribution* for a system with no circulation and one with a great deal of it. This is not a weakness of any particular method; the information is absent from the data. Demonstrating it quantitatively is the project’s headline result.

2 The framework at a glance



Green = built and tested • Orange = your implementation

2.1 Module map

Module	Contents	Status
<code>dfi/fields/domain.py</code>	Box: torus or open box, minimum-image displacement	BUILT
<code>dfi/fields/noise.py</code>	Gaussian random fields, Perlin/fBm, spectral derivatives	BUILT
<code>dfi/fields/base.py</code>	DriftField interface, Helmholtz split, antisymmetric A	BUILT
<code>dfi/fields/random_field.py</code>	GridDriftField, SpectralDriftField	BUILT
<code>dfi/fields/analytic.py</code>	OU, coupled chain, double well, radial+rotational	BUILT
<code>dfi/simulate.py</code>	Euler–Maruyama, numpy + torch/GPU, step diagnostics	BUILT
<code>dfi/trajectories.py</code>	Trajectories container, reshaping, diagnostics	BUILT
<code>dfi/estimators/base.py</code>	Estimator contract, registry, ZeroDrift/OracleDrift	BUILT
<code>dfi/estimators/local.py</code>	Rungs 1–2: binned KM, kernel regression	YOURS
<code>dfi/estimators/basis.py</code>	Rung 3: basis projection (SFI) + error bound	YOURS
<code>dfi/estimators/neural.py</code>	Rung 4: MLP regression	YOURS
<code>dfi/estimators/score.py</code>	Rung 5: KDE score, denoising score matching	YOURS
<code>dfi/metrics.py</code>	Evaluation sets, error decomposition, error vs density	BUILT
<code>dfi/sweeps.py</code>	Config grid runner with caching and resume	BUILT
<code>dfi/viz/</code>	Style tokens, LIC textures, all plots and animations	BUILT

3 Data you can create

This is the part of the framework you drive directly. Three kinds of object: **fields** (the ground truth), **trajectories** (time-ordered observations), and **snapshots** (unordered samples).

3.1 The construction behind every random field

All random fields are built from a *scalar* potential, never from two independent noise vector fields:

$$b(x) = -\nabla U(x) + \omega A \nabla U(x) \quad A^T = -A, \quad A \text{ constant.} \quad (2)$$

Why this exact form. Two identities hold *pointwise and exactly*:

$$\nabla \cdot (A \nabla U) = A_{ij} \partial_i \partial_j U = 0, \quad \nabla U \cdot A \nabla U = 0,$$

the first because an antisymmetric matrix contracted with the symmetric Hessian vanishes, the second because an antisymmetric quadratic form vanishes. So the second term is genuinely divergence-free *and everywhere* orthogonal to $\nabla \rho_{ss}$. Substituting $\rho = e^{-U/D}$ into Fokker–Planck leaves the residual current $J = \omega (A \nabla U) \rho$, which is divergence free.

Consequences, all free:

- the Helmholtz decomposition is *exact*, not estimated;
- $\rho_{ss} = e^{-U/D}/Z$ is exact **for every** ω (verified: Fokker–Planck residual 2×10^{-7});
- `field.with_omega(w)` changes the dynamics while holding ρ_{ss} *bit-identical* — the airtight identifiability experiment;
- ω reads directly as the rotational-to-gradient amplitude ratio;
- it works in *any* dimension (in 2D, A is the perpendicular gradient);
- in $d = 1$, A is necessarily zero — correctly, since no divergence-free field exists on a line.

Helmholtz decomposition -- random-grf-2D

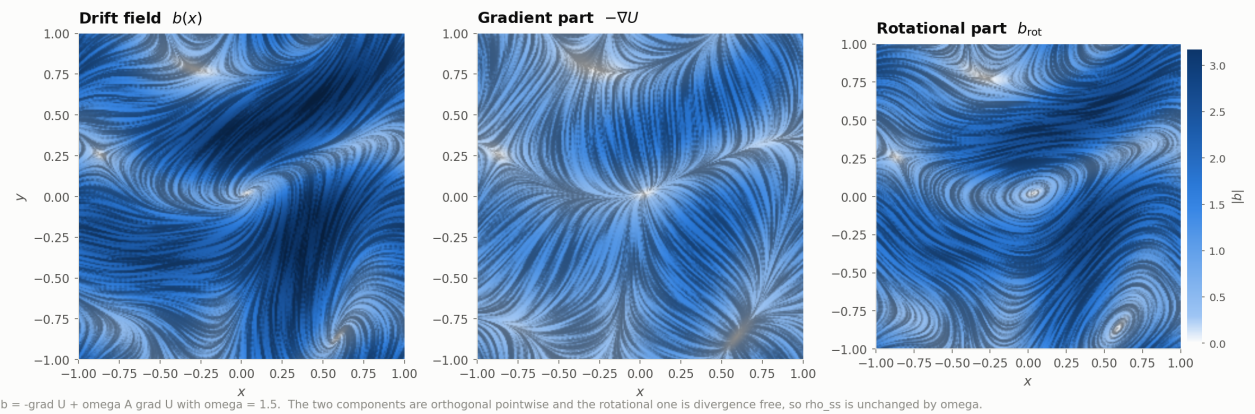


Figure 1: The decomposition of Eq. (2) at $\omega = 1.5$, drawn with line-integral-convolution texture. Left: the full drift. Middle: the gradient part, with a visible source/sink at the origin. Right: the rotational part, circulating. All three share one speed scale.

3.2 Field constructors

Constructor	Dim	What it is
random_grid_field	$d \leq 3$	Potential on a periodic grid; spectral derivatives, quintic-spline interpolation. The backend you <i>draw and animate</i> .
random_spectral_field	any d	Random Fourier features $U = \sum_m a_m \cos(w_m \cdot x + \phi_m)$. Mesh-free, analytic derivatives, runs on GPU. The backend for the d -sweep.
ornstein_uhlenbeck	any d	$b = -kx$ (optionally $+\omega Ax$). Closed-form everything. <i>Validate here first</i> .
coupled_ou_chain	any d	Harmonically coupled chain, $K = kI + cL$. Stiff spectrum, pure gradient.
DoubleWell1D	1	$U = ax^4 - bx^2$. Metastable, non-Gaussian, gradient.
RadialRotational2D	2	Radial U plus $\omega(-y, x)$. Textbook identifiability demonstrator.

Key parameters

Parameter	Default	Meaning and effect
d	2	Dimension. Grid backend dies above 3 (n^d cells).
resolution	256	Grid points per axis (grid backend).
n_features	256	Fourier modes M (spectral backend); 256–1024 is plenty to $d = 10$.
omega	0.0	Rotational strength. 0 = equilibrium; 1 = equal power in both parts.
correlation_length	$L/5$	Feature size. Sets the gradient scale ℓ .
drift_scale	1.0	Rescales U so that $\text{RMS} b $ equals this. Fixes units.
seed	None	Reproducibility.
generator	spectral	'spectral' (Gaussian random field) or 'perlin' (fBm).
kernel	gaussian	'gaussian' (RBF) or 'matern' (rougher), spectral backend.
box	torus	<code>Box.cube(d, 1.0, periodic=True)</code> by default.

Noise spectra (grid backend)

spectrum	$P(k)$	Character
'gaussian'	$e^{-(k\ell)^2/2}$	Default. One smooth scale, all derivatives governed by ℓ , resolution-independent.
'powerlaw'	$(1 + (k\ell)^2)^{-s/2}$	Multi-scale / fBm-like. Use $s \geq 6$ in 2D — see the warning below.
'bandpass'	shell at k_0	Cellular, wave-like structure.

Why the default is Gaussian, not power-law. With $s = 4$ in 2D the variance of the *second* derivatives goes like $\int k^3 P(k) dk$, which diverges logarithmically. The field’s gradient scale $\ell = |b|/|\nabla b|$ is then set by wherever the band limit happens to sit, not by `correlation_length`. Measured: ℓ fell from 0.055 to 0.033 when the grid was refined $128 \rightarrow 256$ — i.e. the “ground truth” depended on grid resolution, and carried real structure at scales no estimator could ever resolve. A Gaussian spectrum cuts off exponentially and held $\ell \approx 0.24$ at both resolutions.

3.3 Domains

`Box(lo, hi, periodic)` with helpers `Box.cube(d, half_width, periodic)` and `Box.unit(d, length, periodic)`. Boundary modes for simulation: ‘periodic’ (wrap), ‘reflecting’ (mirror), ‘none’ (open).

The torus is the default, and `Box.displacement` is why it works. On a periodic box no walker escapes and the whole domain gets sampled, which removes the empty-tail-bin pathology from the estimator comparison. The cost is one piece of bookkeeping, done in exactly one place: a walker that steps off the right edge reappears on the left, so the raw difference $x_{t+1} - x_t$ is a box-sized jump. The minimum-image convention returns the shortest representative instead. *Every* increment in the project goes through `Box.displacement`.

3.4 Trajectories: `simulate()`

Parameter	Default	Meaning
<code>n_walkers</code>	256	N independent walkers.
<code>n_steps</code>	1000	T recorded transitions ($T + 1$ frames).
<code>dt</code>	0.01	Observation interval — the spacing of recorded frames.
<code>D</code>	0.05	Diffusion coefficient; noise amplitude $\sqrt{2D dt}$.
<code>substeps</code>	1	Euler sub-steps per recorded interval (integration at dt/k).
<code>burn_in</code>	0.0	Time run and discarded, so frames come from the stationary state.
<code>x0</code>	None	None (uniform on torus), a point $(d,)$ (all together — the spreading-cloud run), an array (N,d) , or ‘stationary’/‘uniform’/‘center’.
<code>obs_noise</code>	0.0	Per-coordinate measurement noise added to recorded positions.
<code>boundary</code>	auto	‘periodic’, ‘reflecting’, ‘none’.
<code>backend</code>	numpy	‘numpy’ or ‘torch’ (GPU; needs <code>field.supports_torch</code>).
<code>torch_dtype</code>	float32	float32 is $\sim 2\times$ faster on consumer GPUs.
<code>store_dtype</code>	float32	Storage precision; integration is always float64.

Two different Δt ’s, and confusing them ruins the bias experiment. `dt` is what the *observer* sees; `dt/substeps` is what the *integrator* uses. With `substeps=1` an $O(\Delta t)$ error in the estimator is indistinguishable from an $O(\Delta t)$ error in the simulator, and you would be measuring your own integrator. Simulate with `substeps` large, observe at `dt`, and the bias you plot is the estimator’s.

Step-size diagnostics

`suggest_dt(field, D)` returns the largest Δt meeting three criteria at once; `check_step(field, dt, D)` reports them:

Key	Quantity	Want
<code>drift_length_scale</code>	$\ell = b / \nabla b $	the field's own gradient scale
<code>drift_cfl</code>	$ b \Delta t / \ell$	< 0.1
<code>diffusive_cfl</code>	$\sqrt{2dD\Delta t}/\ell$	< 0.2 — usually the binding one
<code>stability</code>	$ \nabla b \Delta t$	< 0.1 ; above ~ 1 the run is nonsense
<code>ok</code>	all three	<code>simulate</code> warns if <code>False</code>

The diffusive criterion is the one people get wrong: a walker that diffuses a full gradient scale per step steps straight *over* the structure of b , and the stationary density comes out visibly wrong even though the run looks perfectly stable.

3.5 What a `Trajectories` object holds

Attribute	Shape / type	Notes
<code>.x</code>	$(N, T+1, d)$	observed positions (noisy if <code>obs_noise > 0</code>)
<code>.x_true</code>	$(N, T+1, d)$	clean positions, kept only when noise was added
<code>.dt, .D</code>	float	observation interval, diffusion coefficient
<code>.box</code>	Box	carries the periodicity increments need
<code>.dt_integration</code>	float	<code>dt/substeps</code>
<code>.n_transitions</code>	int	$N \cdot T$ — the sample size that governs error
<code>.times, .duration</code>	array, float	

Operations

Method	Returns / purpose
<code>.increments()</code>	(N, T, d) displacements, minimum-image corrected
<code>.pairs()</code>	(x, dx) flattened to $(N \cdot T, d)$ — the regression data
<code>.drift_target()</code>	$(x, dx/dt)$ — the Kramers–Moyal target
<code>.subsample_time(k)</code>	every k -th frame, $\Delta t \rightarrow k\Delta t$ — the Δt sweep
<code>.subsample_walkers(n)</code>	the N axis of a sweep
<code>.truncate(n_frames)</code>	the T axis of a sweep
<code>.with_observation_noise(σ)</code>	add measurement noise after the fact
<code>.msd()</code>	mean squared displacement per frame
<code>.step_statistics()</code>	dict: rms step vs pure-diffusion prediction, etc.
<code>.save()</code> / <code>.load()</code>	compressed <code>.npz</code> round-trip

`subsample_time` runs a Δt sweep over the *same underlying paths*, so differences across the sweep are pure estimator bias rather than a new realisation. Simulate once, sweep for free.

3.6 Snapshots

`sample_stationary(field, n, D, method=...)` returns (n, d) independent draws from ρ_{ss} with *no time ordering* — the input to the density-only half of the project.

- `method='grid'` — samples the exact Boltzmann density on the field's own grid. Fast, no equilibration error. Verified to add no bias beyond the independent-sample floor.
- `method='dynamic'` — runs independent walkers past equilibrium and takes the final frame. Works for any field, including high d .

3.7 Measurement noise

Noise is applied *after* simulation, which is the physically right order: the walker follows the true dynamics and the microscope is what is imprecise.

The $\sigma/\Delta t$ pathology. Observation noise adds $2\sigma^2$ to the increment variance — a term that does **not** vanish as $\Delta t \rightarrow 0$. The apparent drift $\Delta x/\Delta t$ therefore picks up a spurious contribution $\sim \sigma/\Delta t$, so *finer sampling makes things worse*. This is the actual limiting factor in real single-particle-tracking data, and it is why `obs_noise` is a first-class sweep axis rather than an afterthought.

3.8 A field's own accessors

Every field answers these, which is what makes the metrics exact:

Method	Returns
<code>.drift(x)</code>	$b(x)$, shape $(\dots, d)$
<code>.potential(x)</code>	$U(x)$
<code>.grad_potential(x)</code>	$\nabla U(x)$
<code>.gradient_part(x)</code>	$-\nabla U$
<code>.rotational_part(x)</code>	$b + \nabla U$
<code>.log_rho_ss(x, D)</code>	$\log \rho_{ss}$ up to a constant
<code>.score_ss(x, D)</code>	$\nabla \log \rho_{ss}$
<code>.stationary_current(x, D)</code>	$J = b\rho - D\nabla\rho$; zero iff detailed balance
<code>.divergence(x)</code>	$\nabla \cdot b$ (exact where available)
<code>.typical_scale()</code>	$\text{RMS} b $ — the unit errors are reported in
<code>.concentration(D)</code>	how tightly ρ_{ss} concentrates (see below)
<code>.with_omega(w)</code>	same U , different circulation
<code>.entropy_production(D)</code>	$\sigma = \langle b_{\text{rot}} ^2 \rangle / D$, by quadrature

Choose D deliberately. `field.concentration(D)` reports the density's dynamic range. Below ~ 10 the walkers cover the whole domain and the estimator comparison is about the *method*. Above ~ 1000

they sit in one basin, most of the field is never visited, and any “error over the box” is really a statement about extrapolation. Picking D by eye from a picture is how people end up comparing estimators on data that never visited most of the field.

4 Estimators

4.1 The contract

Everything downstream — metrics, sweeps, phase diagrams — talks to estimators only through this interface. Implement it and the entire comparison machinery works on your method with no further wiring.

```
class TrajectoryEstimator(DriftEstimator):    # sees (x, dx): time ordering
    def fit(self, traj: Trajectories) -> Self: ...
    def predict(self, x) -> np.ndarray:      ... # (... , d) in and out
    def support(self, x) -> np.ndarray:      ... # where is it trustworthy?
    def uncertainty(self, x):                ... # optional: own error bar
    def predict_diffusion(self, x):          ... # optional: D(x)

class SnapshotEstimator(DriftEstimator):     # sees positions only
    def fit(self, samples, D, box=None) -> Self: ...
    def predict_score(self, x) -> np.ndarray: ... # grad log rho_ss
    # predict() = D * predict_score(), so the equilibrium assumption
    # is written down in exactly one place instead of being smuggled in
```

Two helpers on the trajectory base class: `self.regression_data(traj)` gives $(x, \Delta x/\Delta t)$, and `self.noise_level(traj)` gives $\sqrt{2D/\Delta t}$ — the per-coordinate noise on each raw target.

Why every rung is a form of local averaging. The raw pointwise target $\Delta x/\Delta t$ has standard deviation $\sqrt{2D/\Delta t}$, which for typical settings is an *order of magnitude larger* than $|b|$ itself. In the shipped demo, $\text{RMS}|b| = 1.0$ while $\text{RMS}|\Delta x/\Delta t| = 17.5$. That is not a defect to be fixed; it is the whole problem. Every method in the ladder differs only in *how* it averages that noise away.

4.2 The ladder

#	Class	Key	Input	Expected behaviour
1	BinnedKramersMoyal	binmed_km	trajectories	Cheapest, noisiest. Undefined in high d (bins ^{d} cells).
2	KernelRegression	kernel	trajectories	Smooth weights; local-linear removes the density-gradient bias.
3	BasisProjection	sfi	trajectories	Expected winner in low d. Closed form + analytic error bound.
4	NeuralDrift	nn	trajectories	Should lose in $d \leq 2$; wins once the basis outgrows the data.
5a	KDEScore	kde_score	snapshots	Recovers the gradient part only — by construction.
5b	DenoisingScoreMatching	dsm	snapshots	Same, learned. The diffusion-model hook.
–	ZeroDrift	zero	–	Built. Null model; scores exactly 1.0.
–	OracleDrift	oracle	–	Built. Truth; must score ≈ 0 .

4.3 Rung 1 — Binned Kramers–Moyal

$$\hat{b}(x) = \frac{1}{\Delta t} \frac{\sum_{i \in \text{cell}(x)} \Delta x_i}{\#\{i \in \text{cell}(x)\}}.$$

Vectorise it: `np.ravel_multi_index` turns a d -dimensional cell address into a flat one, then `np.bincount(flat, weights=dx[:,j])` sums each component in one pass. The whole estimator is three

lines; looping over cells is $100\times$ slower.

What to expect and report: variance falling as $n(x)^{-1/2}$ per cell; an $O(h^2)$ bias from averaging b over a cell of width h ; therefore a U-shaped total error with an optimal bin count. In $d = 6$ with 32 bins there are 10^9 cells for maybe 10^7 samples — the estimator is not merely inaccurate but *undefined*, which is the honest reason the ladder continues.

4.4 Rung 2 — Kernel regression

$$\hat{b}(x) = \frac{\sum_i K_h(x - x_i) \Delta x_i / \Delta t}{\sum_i K_h(x - x_i)}.$$

That is Nadaraya–Watson; `local_linear=True` instead fits a local linear model by weighted least squares and keeps the constant term.

Why the local-linear variant matters. Nadaraya–Watson is badly biased wherever the sample density has a strong gradient: the kernel pulls in more samples from the dense side, dragging the average toward that side’s drift value. Since the density here is $e^{-U/D}$, its gradient is largest exactly on the slopes where the drift is largest. Local-linear fitting cancels that term. Demonstrating this on the double well, where the density gradient is savage, is a genuinely good result.

Speed. Naive evaluation is $O(n_{\text{query}} n_{\text{samples}})$. Use a `ckdTree` with radius $\sim 3h$, or bin to a fine grid and convolve with `gaussian_filter` — the latter turns the estimator into two convolutions and is exact for gridded queries.

4.5 Rung 3 — Basis projection (Stochastic Force Inference)

Expand $b_j(x) = \sum_a c_{ja} f_a(x)$ and solve by least squares against the same Kramers–Moyal targets:

$$C = (F^\top F)^{-1} F^\top Y, \quad Y = \Delta x / \Delta t.$$

Every increment informs every coefficient, so instead of dividing the data among bins you use all of it for each unknown. Reference: Frishman & Ronceray, *Phys. Rev. X* **10**, 021009 (2020).

Implement the information bound — it is the best part. The method estimates its own error from the data alone, with no ground truth:

$$\langle |\hat{b} - b|^2 \rangle \approx \frac{n_{\text{basis}}}{n} \frac{d}{\Delta t} D + (\text{projection bias}).$$

This turns “my error is 4%” into “my error is 4%, the theory says 3.7%, and here they are on the same axes across three decades of sample size”. An error bar that needs no ground truth is what makes a method usable on real data.

Bases. Polynomial (degree 1 spans OU *exactly* — the ideal validation case); Fourier (natural on the torus, and the random fields *are* band-limited sums of exactly this form, so the bias term is genuinely zero); RBF.

The curse, quantified. Polynomials of degree p in d dimensions number $\binom{d+p}{p}$: at $p = 3$, $d = 2$ gives 10 but $d = 10$ gives 286. Fourier is far worse, $(2n_{\text{max}} + 1)^d$. Since the variance term grows linearly in n_{basis} , **this is the crossover the phase diagram is looking for.**

4.6 Rung 4 — Neural network regression

An MLP $x \mapsto b(x)$ trained on the same target. There is no trick. The question is not *can* a network do this (it can) but *where it starts to be worth it*.

Do not judge convergence by the loss. The squared loss is dominated by the irreducible noise floor $2dD/\Delta t$ and barely moves even as the fit improves. `self.noise_floor(traj)` returns it; plot the excess over it, or the curve looks flat from epoch one and tells you nothing.

Other things that decide whether this works: standardise inputs *and* targets (targets are $10\text{--}100\times$ larger than the drift); hold out whole *walkers*, not random timepoints (consecutive frames are correlated, so a random split leaks); feed $[\sin(2\pi x/L), \cos(2\pi x/L)]$ on a torus or the network must learn that the two edges are the same place and will show a seam; and sweep the implicit smoothness (width, weight decay) or you are comparing a tuned method against an untuned one.

4.7 Rung 5 — Score-based, from snapshots

The identity that makes this substantive rather than decorative:

$$\nabla \log \rho_{\text{ss}}(x) = -\nabla U(x)/D = b(x)/D \quad (\text{gradient drift only}). \quad (3)$$

A denoising score-matching model trained on snapshots is not a black box borrowed from generative modelling; it is estimating precisely the physical quantity we want.

KDE score has a closed-form gradient, so no numerical differentiation is needed: $\nabla \log \hat{\rho}(x) = -h^{-2} \sum_i w_i (x - x_i) / \sum_i w_i$. Use `box.displacement` for the offsets or the estimate is wrong near the edges. The interesting catch: the optimal bandwidth for estimating ρ is too small for estimating $\nabla \log \rho$, and in the tails the log-gradient is dominated by the few samples that happen to be there.

Denoising score matching minimises $\mathbb{E}[|s_\theta(\tilde{x}, \sigma) + (\tilde{x} - x)/\sigma^2|^2]$ with $\tilde{x} = x + \sigma\epsilon$. Evaluate at the *smallest* σ in the training range. Here σ plays the role of diffusion time in a generative model: training this is the same computation as training a diffusion model on the snapshot dataset — the only difference is that we want the score itself rather than samples from it.

Then run it on a rotational system and watch it fail — as it must. Both snapshot methods return the gradient part with the rotational part missing, for every ω . That confirmation is the point of the whole rung.

5 Measuring and comparing

5.1 Where you evaluate matters more than the metric

`make_eval_set(field, D, measure=...)` makes the choice explicit and records it:

- 'stationary' — points drawn from ρ_{ss} . The default and the honest one: it scores a method where the data actually is.
- 'uniform' — uniform on the box. Harsher. **The gap between the two is the extrapolation penalty.**

Pass `traj=` and each evaluation point also carries the training data's local density, which is what `error_vs_density` bins against.

5.2 What `drift_error` returns

Key	Meaning
<code>nrmse</code>	$\text{RMS} \hat{b} - b /\text{RMS} b $. The headline number. 0 perfect, 1 = as good as predicting zero.
<code>nrmse_grad</code>	error in the <i>gradient</i> direction, normalised by that component
<code>nrmse_rot</code>	error in the <i>rotational</i> direction
<code>nrmse_perp</code>	residual, off both structure directions
<code>cosine</code>	mean alignment. Separates “right shape, wrong scale” from “wrong shape”
<code>scale_ratio</code>	$\text{RMS} \hat{b} /\text{RMS} b $. Below 1 = over-smoothing
<code>supported_fraction</code>	fraction of points the estimator claims to cover

Normalisation is what makes the numbers mean something. Because everything is divided by $\text{RMS}|b|$, `nrmse = 1.0` means “learned nothing”. Methods genuinely do score *above* 1 at small sample sizes, and a reader cannot tell without the null line — which is why `ZeroDrift` is drawn on every plot. **Diagnostics for free.** A low `cosine` means the shape is wrong; a `scale_ratio` far from 1 usually means a missing $1/\Delta t$ or a factor of 2 in D . Those two catch most first-attempt bugs.

5.3 The error decomposition

Because the two parts of Eq. (2) are orthogonal *pointwise*, the error vector splits exactly along the field's own structure:

$$\hat{b} - b = e_{\text{grad}} \hat{g} + e_{\text{rot}} \hat{r} + e_{\perp}, \quad \hat{g} = \frac{\nabla U}{|\nabla U|}, \quad \hat{r} = \frac{A \nabla U}{|A \nabla U|}.$$

This is what turns the identifiability claim into two numbers. A method that recovers the potential perfectly but misses the circulation entirely — which is what *every* snapshot method must do — shows $e_{\text{grad}} \approx 0$ and e_{rot} equal to the full rotational amplitude. That is a diagnosis, not just an error bar.

5.4 Sweeps

`sweep_grid(**axes)` takes the Cartesian product of named axes into a list of `SweepConfigs`; `run_sweep(configs, estimators, cache=...)` runs them and returns a tidy `DataFrame`, one row per (config, estimator).

Axis	Default	What it sweeps
d	2	dimension (grid backend ≤ 3 , spectral above)
n_walkers	512	N
n_steps	2000	T
dt	None	observation interval (None = suggest_dt)
D	0.3	diffusion coefficient
omega	0.0	rotational strength — the identifiability axis
obs_noise	0.0	measurement noise
substeps	1	integration accuracy at fixed observation dt
seed	0	include several — one seed is a number with no uncertainty
measure	stationary	evaluation measure

Three properties that keep the comparison honest: all estimators see exactly the same data within a config and are scored on the same points; snapshot estimators get a *matched observation budget* ($N \cdot T$ independent samples, the generous reading); and an estimator whose `fit` raises `NotImplementedError` yields a row with `status='not_implemented'` and the sweep continues. Results are cached by config hash, so runs resume and adding a new estimator does not recompute trajectories.

6 Figures and animations

Everything shares one design system (`dfi/viz/style.py`): a single-hue sequential blue ramp for magnitudes, diverging blue↔red with a neutral grey midpoint for signed quantities, a fixed categorical order for method identity, and a light/dark theme switch (`use_style('light'|'dark')`).

Function	Draws
<i>Fields</i>	
<code>plot_drift</code>	the field, as LIC texture / streamlines / quiver
<code>plot_potential</code>	U heatmap with contours (diverging ramp, zero at grey)
<code>plot_density</code>	ρ_{ss} , exact or empirical
<code>plot_helmholtz</code>	the three-panel decomposition, shared colour scale
<code>plot_field_card</code>	four-panel identity card: drift, U , ρ , power spectrum
<code>plot_field_1d</code>	drift, potential and density for 1D systems
<i>Trajectories</i>	
<code>plot_trajectories</code>	paths over the field, colour ramping with time
<code>plot_snapshot</code>	the unordered scatter
<code>plot_sampling_density</code>	where the data actually is
<code>plot_msd</code>	MSD against the free-diffusion line
<code>plot_increment_check</code>	increments vs $\mathcal{N}(0, 2D\Delta t)$
<code>plot_trajectories_1d</code>	x vs t beside the occupancy histogram
<i>Comparison</i>	
<code>plot_error_curves</code>	error vs any sweep axis, with null line and slope guide
<code>plot_identifiability</code>	gradient vs rotational error against ω
<code>plot_phase_diagram</code>	which method wins over (budget, d), with crossover
<code>plot_error_vs_density</code>	error against local sample density
<code>plot_bias_variance</code>	the U-shape against a smoothing parameter
<code>plot_training_curve</code>	NN loss <i>relative to the noise floor</i>
<i>Animations (mp4 via ffmpeg, gif via Pillow)</i>	
<code>animate_diffusion</code>	walkers with fading trails and glowing heads
<code>animate_omega_sweep</code>	ω rising while ρ_{ss} refuses to move
<code>animate_relaxation</code>	a point source spreading into ρ_{ss}

`plot_sampling_density` is the one people skip and shouldn't. A Kramers–Moyal estimate is a local average, so its variance at x scales like $1/n(x)$, and the regions where a method appears to “fail” are usually just the regions nothing visited. Plotting error without plotting sample density beside it invites the wrong conclusion.

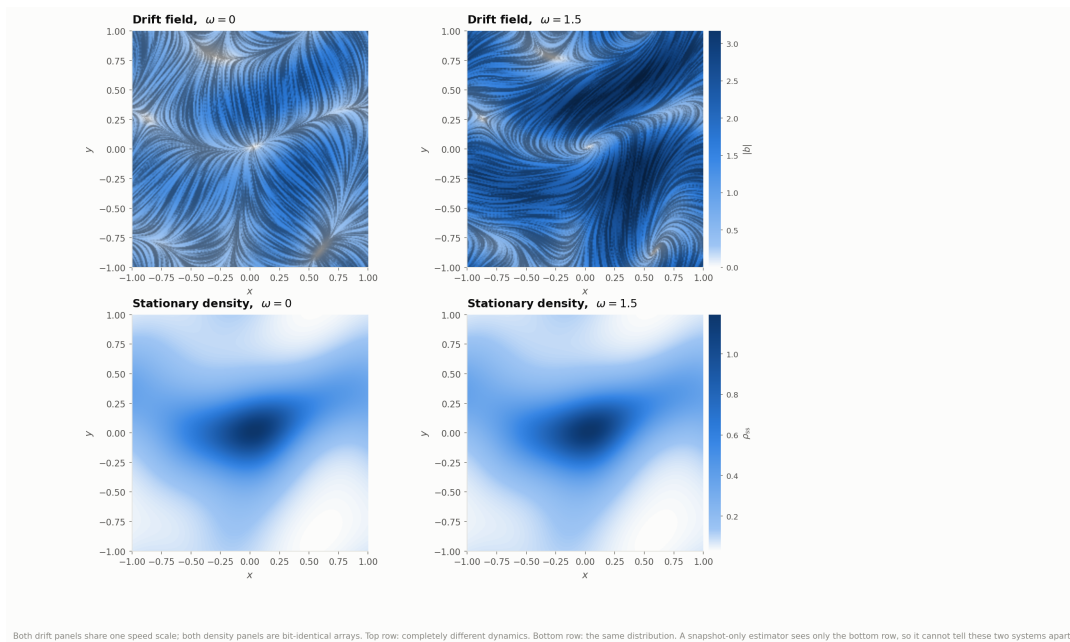


Figure 2: The headline figure. Both drift panels share one speed scale; both density panels are bit-identical arrays. Top row: completely different dynamics. Bottom row: the same distribution. A snapshot-only estimator sees only the bottom row.

7 Design decisions, and the measurements behind them

Decision	Why, with the number
Scalar construction, Eq. (2)	Exact Helmholtz split and exact Boltzmann density for every ω . Fokker–Planck residual: 2×10^{-7} .
Quintic B-splines, band limit at 0.25 Nyquist	Ground-truth drift error 7×10^{-6} relative. Cubic at the same band limit gives 5×10^{-4} ; a 0.55 band limit gives 3×10^{-3} . The estimators being benchmarked have percent-level errors, so the truth must be orders of magnitude tighter.
Gaussian spectrum by default	Power-law $s = 4$ made ℓ fall $0.055 \rightarrow 0.033$ when the grid refined $128 \rightarrow 256$ — a resolution-dependent ground truth. Gaussian holds $\ell \approx 0.24$ at both.
Observation $dt \neq$ integration step	So the measured $O(\Delta t)$ bias is the estimator’s, not the integrator’s.
Periodic torus + minimum image	Whole domain sampled; no empty-tail-bin pathology confounding the comparison.
Spectral tensor backend for $d > 3$	A grid is n^d . Random Fourier features are mesh-free with analytic derivatives.
float32 on GPU	$d = 8$, $N = 40\,000$: numpy 258 s vs torch fp32 13.2 s (19.6×). fp32 is $\sim 2\times$ faster than fp64 on consumer GPUs, at a rounding error five orders of magnitude below the diffusive step.

7.1 Validation already performed

The scaffolding was checked end-to-end with throwaway estimators (kept out of the repository — the real ones are yours). Everything landed on theory:

Check	Expected	Measured
Error scaling with sample budget	ratio 2.00 per $4\times$ data	2.08, 1.92, 2.03
Error vs local sample density	slope -0.5	-0.477
Bias–variance in bin width	U-shape	minimum at 8 bins
Oracle estimator	0	$< 10^{-12}$
Null estimator	1.0	1.0000
Snapshot method, gradient error	constant in ω	0.176962 at <i>every</i> ω
Snapshot method, rotational error	exactly 1.0	1.0068, 1.0016, 1.0007

A lesson worth keeping. Three separate test “failures” during the build were tolerances set below the Monte Carlo standard error, not bugs. The MSD estimator’s relative standard error is $\sqrt{2/N}$ independent of t — 2.6% at $N = 3000$. Derive test tolerances from the sample size; do not guess them.

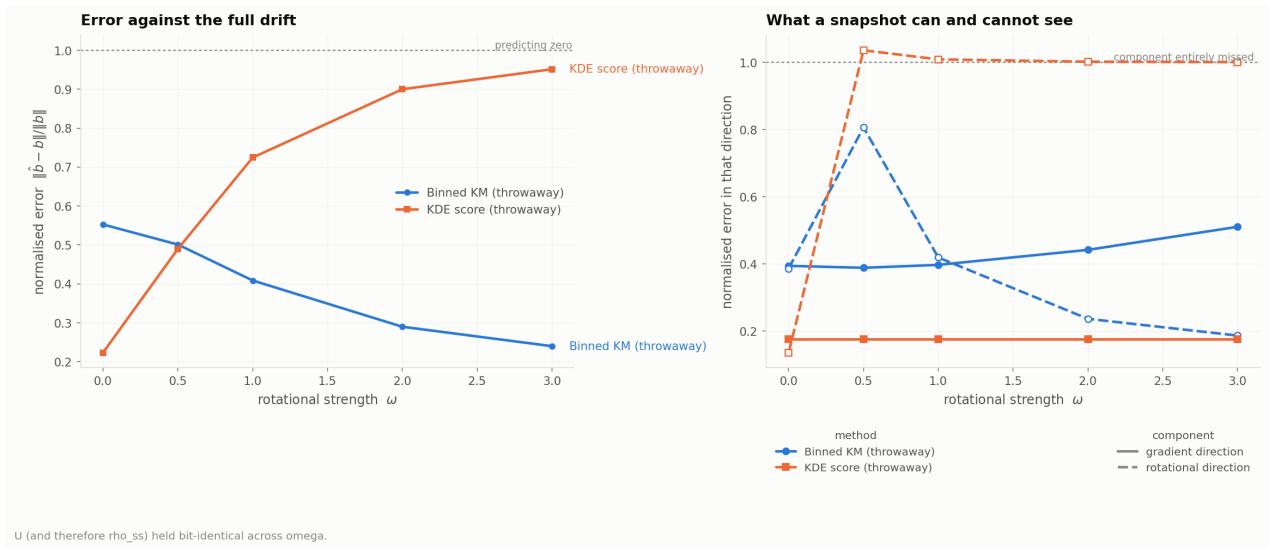


Figure 3: The identifiability result, produced by the built harness. Left: the snapshot method climbs toward the null line as circulation grows, while the trajectory method improves. Right: the decomposition — the snapshot method’s gradient-direction error is flat and its rotational-direction error is pinned at 1.0, meaning it misses *exactly* 100% of the circulation.

8 Cheat sheet

End to end, today

```

from dfi import (random_grid_field, random_spectral_field, simulate,
                 sample_stationary, suggest_dt, check_step)
from dfi.metrics import make_eval_set, drift_error, error_vs_density
from dfi.viz import use_style, plot_drift, plot_helmholtz, plot_trajectories

use_style("light")                                # or "dark"

# 1. a random field on a periodic torus
field = random_grid_field(d=2, resolution=256, omega=1.5, seed=0,
                          correlation_length=0.45, drift_scale=1.0)
print(field.summary(), field.concentration(D=0.3))

# 2. check the step, then simulate
D = 0.3
dt = suggest_dt(field, D)
print(check_step(field, dt, D))
traj = simulate(field, n_walkers=2000, n_steps=4000, dt=dt, D=D,
                burn_in=10.0, substeps=1, seed=0)

# 3. the same potential without circulation -- rho_ss is bit-identical
equil = field.with_omega(0.0)

# 4. snapshots: positions only, no time ordering
snaps = sample_stationary(field, 200_000, D)

# 5. sweep axes for free, over the same underlying paths
coarse = traj.subsample_time(4)                    # dt -> 4*dt
fewer = traj.subsample_walkers(256)                # the N axis
noisy = traj.with_observation_noise(0.01)          # the sigma axis

# 6. fit your estimator and score it
est = MyEstimator().fit(traj)
ev = make_eval_set(field, D, n=20000, measure="stationary", traj=traj)
print(drift_error(est, field, ev))                  # nrmse, nrmse_grad, nrmse_rot, ...

```

High dimension (GPU)

```

field = random_spectral_field(d=8, n_features=512, omega=0.4, seed=0)
traj = simulate(field, n_walkers=40_000, n_steps=2000,
                dt=suggest_dt(field, 0.1), D=0.1,
                backend="torch", torch_dtype="float32")  # 19.6x faster

```

Sweeps and figures

```

from dfi.sweeps import sweep_grid, run_sweep
from dfi.viz.sweeps import plot_error_curves, plot_identifiability

cfgs = sweep_grid(d=2, n_walkers=[64, 256, 1024], n_steps=2000,
                  omega=[0.0, 1.0, 2.0], seed=[0, 1, 2])
df = run_sweep(cfgs, {"mine": MyEstimator}, cache="outputs/data/sweep.csv")

plot_error_curves(df, x="n_transitions", y="nrmse")

```

```
plot_identifiability(df)
```

Scripts and tests

Command	Does
<code>python scripts/01_fields_and_diffusion.py</code>	all static figures (~ 1 min)
<code>... --animate</code>	plus mp4 animations (~ 5 min)
<code>... --theme dark</code>	dark theme
<code>python scripts/02_sweeps.py --quick</code>	sweep figures; works with baselines alone
<code>pytest -q</code>	28 tests; skips unimplemented rungs
<code>pytest -q -k binned_km</code>	one rung, verdict in ~ 2 s

Suggested order of work. Implement `BinnedKramersMoyal.fit`, run `pytest -q -k binned_km` until it recovers $b(x) = -kx$ on 1D OU, then `scripts/02_sweeps.py --quick` to see your first real curve. Work up the ladder from there. Each rung is a genuine baseline for the next, and the negative results — where the simple method wins — are the valuable output.